

Inheritance, Polymorphism, Exception handling and file handling

Inheritance

1) Create a class called Person.

Populate the class with below private fields

- 1. FirstName as string**
- 2. LastName as string**
- 3. EmailAddress as string**
- 4. DateOfBirth as DateTime**

Add a constructor that accept FirstName, LastName, Email, DateOfBirth as parameters

Add read-only properties that return the following computed information:

- 1. IsAdult as bool whether or not the person is over 18**
- 2. SunSign as string the traditional western sun sign of the person**
- 3. IsBirthDay as bool whether or not today is the person's birthday**
- 4. ScreenName as string a default screen name that you might see being offered to a first time user of Gmail or Yahoo (e.g. Hari Joe born on May 25th, 1980 might be hdoe525 or haridoe052580)**

Create a class classed Employee which should be derived from Person class. Employee class should add the following properties:

- 1. Salary as double**

In the Main method create object of Employee class and test the behaviour by calling read-only properties

using System;

class Person

{

```
private string firstName;

private string lastName;

private string emailAddress;

private DateTime dateOfBirth;

public Person(string firstName, string lastName, string emailAddress,
DateTime dateOfBirth)
{
    this.firstName = firstName;

    this.lastName = lastName;

    this.emailAddress = emailAddress;

    this.dateOfBirth = dateOfBirth;
}

public bool IsAdult
{
    get
    {
        int age = DateTime.Now.Year - dateOfBirth.Year;

        if (dateOfBirth > DateTime.Now.AddYears(-age))
            age--;

        return age >= 18;
    }
}

public string SunSign
{
    get
```

```
{  
    int day = dateOfBirth.Day;  
    int month = dateOfBirth.Month;  
    if ((month == 3 && day >= 21) || (month == 4 && day <= 19))  
        return "Aries";  
    else if ((month == 4 && day >= 20) || (month == 5 && day <= 20))  
        return "Taurus";  
    else if ((month == 5 && day >= 21) || (month == 6 && day <= 20))  
        return "Gemini";  
    else if ((month == 6 && day >= 21) || (month == 7 && day <= 22))  
        return "Cancer";  
    else if ((month == 7 && day >= 23) || (month == 8 && day <= 22))  
        return "Leo";  
    else if ((month == 8 && day >= 23) || (month == 9 && day <= 22))  
        return "Virgo";  
    else if ((month == 9 && day >= 23) || (month == 10 && day <= 22))  
        return "Libra";  
    else if ((month == 10 && day >= 23) || (month == 11 && day <= 21))  
        return "Scorpio";  
    else if ((month == 11 && day >= 22) || (month == 12 && day <= 21))  
        return "Sagittarius";  
    else if ((month == 12 && day >= 22) || (month == 1 && day <= 19))  
        return "Capricorn";  
    else if ((month == 1 && day >= 20) || (month == 2 && day <= 18))  
        return "Aquarius";  
}
```

```

        else
            return "Pisces";
    }
}

public bool IsBirthDay
{
    get
    {
        return (DateTime.Now.Day == dateOfBirth.Day &&
            DateTime.Now.Month == dateOfBirth.Month);
    }
}

public string ScreenName
{
    get
    {
        return firstName.ToLower() +
            lastName.ToLower() +
            dateOfBirth.ToString("ddMMyy");
    }
}

public string FirstName => firstName;
public string LastName => lastName;
public string Email => emailAddress;
public DateTime DateOfBirth => dateOfBirth;

```

```

}

class Employee : Person
{
    public double Salary { get; set; }

    public Employee(string firstName, string lastName, string email,
        DateTime dob, double salary)
        : base(firstName, lastName, email, dob)
    {
        Salary = salary;
    }
}

class Program
{
    static void Main()
    {
        Employee emp = new Employee(
            "Nandini",
            "Kondaveeti",
            "23BQ1A05B2@vvit.net",
            new DateTime(2005, 6, 20),
            50000);

        Console.WriteLine("Employee Details");
        Console.WriteLine("-----");
        Console.WriteLine("First Name   : " + emp.FirstName);
        Console.WriteLine("Last Name    : " + emp.LastName);
    }
}

```

```
Console.WriteLine("Email      : " + emp.Email);

Console.WriteLine("Date of Birth: " +
emp.DateOfBirth.ToString("dd/MM/yyyy"));

Console.WriteLine("Salary      : " + emp.Salary);

Console.WriteLine("\nComputed Properties");

Console.WriteLine("-----");

Console.WriteLine("Is Adult    : " + emp.IsAdult);

Console.WriteLine("Sun Sign    : " + emp.SunSign);

Console.WriteLine("Is Birthday : " + emp.IsBirthDay);

Console.WriteLine("Screen Name : " + emp.ScreenName);

Console.ReadLine();

}

}
```

OUTPUT

Employee Details

First Name : Nandini

Last Name : Kondaveeti

Email : 23BQ1A05B2@vvit.net

Date of Birth: 20/06/2005

Salary : 50000

Computed Properties

Is Adult : True

Sun Sign : Gemini

Is Birthday : False

Screen Name : nandinikondaveeti200605

2) create a derived class HourlyEmployee, and Permanent Employee from the above

Person class which will have the below properties respectively.

HourlyEmployee

1) HoursWorked as double

2) PayPerHour as double

Permanant Employee

1) HRA as double

2) DA as double

3) Tax as double

4) NetPay as double

5) TotalPay as double

create an instance in Main method and test the classes

using System;

class Person

```
{  
    public Person()  
    {  
    }  
}
```

```
class HourlyEmployee : Person
{
    public double HoursWorked { get; set; }
    public double PayPerHour { get; set; }
    public HourlyEmployee(double hours, double pay)
    {
        HoursWorked = hours;
        PayPerHour = pay;
    }
    public void Display()
    {
        Console.WriteLine("Hourly Employee");
        Console.WriteLine("-----");
        Console.WriteLine("Hours Worked : " + HoursWorked);
        Console.WriteLine("Pay Per Hour : " + PayPerHour);
        Console.WriteLine("Total Pay   : " + (HoursWorked * PayPerHour));
    }
}
```

```
class PermanentEmployee : Person
{
    public double HRA { get; set; }
    public double DA { get; set; }
    public double Tax { get; set; }
    public double NetPay { get; set; }
    public double TotalPay { get; set; }
```



```

public PermanentEmployee(double basicSalary)
{
    HRA = basicSalary * 0.15;
    DA = basicSalary * 0.10;
    TotalPay = basicSalary + HRA + DA;
    Tax = TotalPay * 0.08;
    NetPay = TotalPay - Tax;
}

public void Display()
{
    Console.WriteLine("\nPermanent Employee");
    Console.WriteLine("-----");
    Console.WriteLine("HRA      : " + HRA);
    Console.WriteLine("DA      : " + DA);
    Console.WriteLine("Tax     : " + Tax);
    Console.WriteLine("Total Pay : " + TotalPay);
    Console.WriteLine("Net Pay  : " + NetPay);
}
}

class Program
{
    static void Main()
    {
        HourlyEmployee hourly = new HourlyEmployee(40, 500);
        PermanentEmployee permanent = new PermanentEmployee(50000);
    }
}

```

```
        hourly.Display();  
        permanent.Display();  
        Console.ReadLine();  
    }  
}
```

OUTPUT

Hourly Employee

Hours Worked : 40

Pay Per Hour : 500

Total Pay : 20000

Permanent Employee

HRA : 7500

DA : 5000

Tax : 5000

Total Pay : 62500

Net Pay : 57500

3) Create an interface called IPayable and implement the interface in the above classes (Hourly Employee & PermanentEmployee).

```
using System;
```

```
interface IPayable
```

```
{
```

```
        void CalculatePay();
    }

    class Person
    {
    }

    class HourlyEmployee : Person, IPayable
    {
        double HoursWorked;

        double PayPerHour;

        double TotalPay;

        public HourlyEmployee(double hoursWorked, double payPerHour)
        {
            HoursWorked = hoursWorked;

            PayPerHour = payPerHour;
        }

        public void CalculatePay()
        {
            TotalPay = HoursWorked * PayPerHour;
        }

        public void Display()
        {
            Console.WriteLine("\nHourly Employee");

            Console.WriteLine("Hours Worked : " + HoursWorked);

            Console.WriteLine("Pay Per Hour : " + PayPerHour);

            Console.WriteLine("Total Pay   : " + TotalPay);
        }
    }
}
```

```

    }
}
class PermanentEmployee : Person, IPayable
{
    double BasicSalary;
    double HRA, DA, Tax, TotalPay, NetPay;
    public PermanentEmployee(double basicSalary)
    {
        BasicSalary = basicSalary;
    }
    public void CalculatePay()
    {
        HRA = BasicSalary * 0.15;
        DA = BasicSalary * 0.10;
        TotalPay = BasicSalary + HRA + DA;
        Tax = TotalPay * 0.08;
        NetPay = TotalPay - Tax;
    }
    public void Display()
    {
        Console.WriteLine("\nPermanent Employee");
        Console.WriteLine("Basic Salary : " + BasicSalary);
        Console.WriteLine("HRA      : " + HRA);
        Console.WriteLine("DA      : " + DA);
        Console.WriteLine("Tax     : " + Tax);
    }
}

```

```

        Console.WriteLine("Total Pay   : " + TotalPay);
        Console.WriteLine("Net Pay    : " + NetPay);
    }
}

class Program
{
    static void Main()
    {
        Console.Write("Enter Hours Worked: ");
        double hours = Convert.ToDouble(Console.ReadLine());
        Console.Write("Enter Pay Per Hour: ");
        double pay = Convert.ToDouble(Console.ReadLine());
        HourlyEmployee h = new HourlyEmployee(hours, pay);
        h.CalculatePay();
        h.Display();

        Console.Write("\nEnter Basic Salary: ");
        double basic = Convert.ToDouble(Console.ReadLine());
        PermanentEmployee p = new PermanentEmployee(basic);
        p.CalculatePay();
        p.Display();
    }
}

```

OUTPUT

Enter Hours Worked: 40

Enter Pay Per Hour: 500

Hourly Employee

Hours Worked : 40

Pay Per Hour : 500

Total Pay : 20000

Enter Basic Salary: 50000

Permanent Employee

Basic Salary : 50000

HRA : 7500

DA : 5000

Tax : 5000

Total Pay : 62500

Net Pay : 57500

4) Declare a Method CalculatePay in the interface

using System;

interface IPayable

{

void CalculatePay();

}

class Person

{

}

class HourlyEmployee : Person, IPayable

{

```

double HoursWorked;

double PayPerHour;

double TotalPay;

public HourlyEmployee(double hoursWorked, double payPerHour)
{
    HoursWorked = hoursWorked;

    PayPerHour = payPerHour;
}

public void CalculatePay()
{
    TotalPay = HoursWorked * PayPerHour;
}

public void Display()
{
    Console.WriteLine("\nHourly Employee");

    Console.WriteLine("Hours Worked : " + HoursWorked);

    Console.WriteLine("Pay Per Hour : " + PayPerHour);

    Console.WriteLine("Total Pay   : " + TotalPay);
}
}

class PermanentEmployee : Person, IPayable
{
    double BasicSalary;

    double HRA, DA, Tax, TotalPay, NetPay;

    public PermanentEmployee(double basicSalary)

```

```

{
    BasicSalary = basicSalary;
}

public void CalculatePay()
{
    HRA = BasicSalary * 0.15;
    DA = BasicSalary * 0.10;
    TotalPay = BasicSalary + HRA + DA;
    Tax = TotalPay * 0.08;
    NetPay = TotalPay - Tax;
}

public void Display()
{
    Console.WriteLine("\nPermanent Employee");
    Console.WriteLine("Basic Salary : " + BasicSalary);
    Console.WriteLine("HRA      : " + HRA);
    Console.WriteLine("DA      : " + DA);
    Console.WriteLine("Tax      : " + Tax);
    Console.WriteLine("Total Pay  : " + TotalPay);
    Console.WriteLine("Net Pay   : " + NetPay);
}
}

class Program
{
    static void Main()

```



```

{
    Console.Write("Enter Hours Worked: ");
    double hours = Convert.ToDouble(Console.ReadLine());
    Console.Write("Enter Pay Per Hour: ");
    double pay = Convert.ToDouble(Console.ReadLine());
    HourlyEmployee h = new HourlyEmployee(hours, pay);
    h.CalculatePay();
    h.Display();

    Console.Write("\nEnter Basic Salary: ");
    double basic = Convert.ToDouble(Console.ReadLine());
    PermanentEmployee p = new PermanentEmployee(basic);
    p.CalculatePay();
    p.Display();
    Console.ReadLine();
}
}

```

OUTPUT

Enter Hours Worked: 40

Enter Pay Per Hour: 500

Hourly Employee

Hours Worked : 40

Pay Per Hour : 500

Total Pay : 20000

Enter Basic Salary: 50000

Permanent Employee

Basic Salary : 50000

HRA : 7500

DA : 5000

Tax : 5000

Total Pay : 62500

Net Pay : 57500

5) Implement the interface in above derieved classes and test the interface implementation by passing some values

using System;

interface IPayable

```
{  
    void CalculatePay();  
}
```

class Person

```
{  
  
}
```

class HourlyEmployee : Person, IPayable

```
{  
    double HoursWorked;  
    double PayPerHour;  
    double TotalPay;  
    public HourlyEmployee(double hoursWorked, double payPerHour)  
    {
```

```

        HoursWorked = hoursWorked;

        PayPerHour = payPerHour;
    }

    public void CalculatePay()
    {
        TotalPay = HoursWorked * PayPerHour;
    }

    public void Display()
    {
        Console.WriteLine("\nHourly Employee");
        Console.WriteLine("Hours Worked : " + HoursWorked);
        Console.WriteLine("Pay Per Hour : " + PayPerHour);
        Console.WriteLine("Total Pay   : " + TotalPay);
    }
}

class PermanentEmployee : Person, IPayable
{
    double BasicSalary;
    double HRA, DA, Tax, TotalPay, NetPay;
    public PermanentEmployee(double basicSalary)
    {
        BasicSalary = basicSalary;
    }

    public void CalculatePay()
    {

```

```

        HRA = BasicSalary * 0.15;
        DA = BasicSalary * 0.10;
        TotalPay = BasicSalary + HRA + DA;
        Tax = TotalPay * 0.08;
        NetPay = TotalPay - Tax;
    }

    public void Display()
    {
        Console.WriteLine("\nPermanent Employee");
        Console.WriteLine("Basic Salary : " + BasicSalary);
        Console.WriteLine("HRA      : " + HRA);
        Console.WriteLine("DA      : " + DA);
        Console.WriteLine("Tax      : " + Tax);
        Console.WriteLine("Total Pay   : " + TotalPay);
        Console.WriteLine("Net Pay    : " + NetPay);
    }
}

class Program
{
    static void Main()
    {
        Console.Write("Enter Hours Worked: ");
        double hours = Convert.ToDouble(Console.ReadLine());
        Console.Write("Enter Pay Per Hour: ");
        double pay = Convert.ToDouble(Console.ReadLine());
    }
}

```

```
IPayable hourly = new HourlyEmployee(hours, pay);  
hourly.CalculatePay();  
((HourlyEmployee)hourly).Display();  
Console.Write("\nEnter Basic Salary: ");  
double basic = Convert.ToDouble(Console.ReadLine());  
IPayable permanent = new PermanentEmployee(basic);  
permanent.CalculatePay();  
((PermanentEmployee)permanent).Display();  
Console.ReadLine();  
}  
}
```

OUTPUT

Enter Hours Worked: 50

Enter Pay Per Hour: 450

Hourly Employee

Hours Worked : 50

Pay Per Hour : 450

Total Pay : 22500

Enter Basic Salary: 45000

Permanent Employee

Basic Salary : 45000

HRA : 6750

DA : 4500

Tax : 4500

Total Pay : 56250

Net Pay : 51750

Polymorphism

1)Write a Program to Implement the Compile time polymorphism using Function Overloading approach. Calculate the area of circle, triangle and rectangle using function overloading

using System;

class Area

```
{  
    public double CalculateArea(double radius)  
    {  
        return 3.14 * radius * radius;  
    }  
    public double CalculateArea(double length, double breadth)  
    {  
        return length * breadth;  
    }  
    public double CalculateArea(double baseValue, double height, int x)  
    {  
        return 0.5 * baseValue * height;  
    }  
}  
class Program  
{
```

```

static void Main()
{
    Area obj = new Area();
    Console.Write("Enter Radius of Circle: ");
    double radius = Convert.ToDouble(Console.ReadLine());
    Console.Write("Enter Length of Rectangle: ");
    double length = Convert.ToDouble(Console.ReadLine());
    Console.Write("Enter Breadth of Rectangle: ");
    double breadth = Convert.ToDouble(Console.ReadLine());
    Console.Write("Enter Base of Triangle: ");
    double baseValue = Convert.ToDouble(Console.ReadLine());
    Console.Write("Enter Height of Triangle: ");
    double height = Convert.ToDouble(Console.ReadLine());
    Console.WriteLine("\nArea of Circle   : " + obj.CalculateArea(radius));
    Console.WriteLine("Area of Rectangle : " + obj.CalculateArea(length,
breadth));
    Console.WriteLine("Area of Triangle : " + obj.CalculateArea(baseValue,
height, 1));
    Console.ReadLine();
}
}

```

OUTPUT

Enter Radius of Circle: 6

Enter Length of Rectangle: 6

Enter Breadth of Rectangle: 3

Enter Base of Triangle: 6

Enter Height of Triangle: 6

Area of Circle : 113.04

Area of Rectangle : 18

Area of Triangle : 18

2) Write a Program to Implement the Compile Time Polymorphism

a. Write a user defined Math class that implements the overloaded method to implement the mathematical functions.

Return Type Method

Int Add(inta, int b)

Int Add(inta, int b, int c)

b. Similarly implement the method overloading for Multiply, divide and subtract functionalities.

using System;

class Math

{

public int Add(int a, int b)

{

return a + b;

}

public int Add(int a, int b, int c)

{

return a + b + c;

}


```
public int Subtract(int a, int b)
{
    return a - b;
}

public int Subtract(int a, int b, int c)
{
    return a - b - c;
}

public int Multiply(int a, int b)
{
    return a * b;
}

public int Multiply(int a, int b, int c)
{
    return a * b * c;
}

public int Divide(int a, int b)
{
    return a / b;
}

public int Divide(int a, int b, int c)
{
    return a / b / c;
}
}
```

```

class Program
{
    static void Main()
    {
        Math obj = new Math();

        Console.Write("Enter First Number: ");

        int a = Convert.ToInt32(Console.ReadLine());

        Console.Write("Enter Second Number: ");

        int b = Convert.ToInt32(Console.ReadLine());

        Console.Write("Enter Third Number: ");

        int c = Convert.ToInt32(Console.ReadLine());

        Console.WriteLine("\nAddition (2 Numbers)    : " + obj.Add(a, b));

        Console.WriteLine("Addition (3 Numbers)    : " + obj.Add(a, b, c));

        Console.WriteLine("Subtraction (2 Numbers) : " + obj.Subtract(a, b));

        Console.WriteLine("Subtraction (3 Numbers) : " + obj.Subtract(a, b, c));

        Console.WriteLine("Multiplication (2 Numbers): " + obj.Multiply(a, b));

        Console.WriteLine("Multiplication (3 Numbers): " + obj.Multiply(a, b, c));

        Console.WriteLine("Division (2 Numbers)    : " + obj.Divide(a, b));

        Console.WriteLine("Division (3 Numbers)    : " + obj.Divide(a, b, c));

        Console.ReadLine();

    }
}

```

OUTPUT

Enter First Number: 9

Enter Second Number: 3

Enter Third Number: 6

Addition (2 Numbers) : 12

Addition (3 Numbers) : 18

Subtraction (2 Numbers) : 6

Subtraction (3 Numbers) : 0

Multiplication (2 Numbers): 27

Multiplication (3 Numbers): 162

Division (2 Numbers) : 3

Division (3 Numbers) : 0

Exception

1)Write a Program to handle FormatException. if a user enters values other than integer while entering the marks of each subject, catch FormatException. The name of the students and marks in 3 subjects are taken from the user while executing the program.

```
using System;
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        try
```

```
        {
```

```
            Console.Write("Enter Student Name: ");
```

```
            string name = Console.ReadLine();
```

```

        Console.Write("Enter Marks in Subject 1: ");
        int m1 = Convert.ToInt32(Console.ReadLine());
        Console.Write("Enter Marks in Subject 2: ");
        int m2 = Convert.ToInt32(Console.ReadLine());
        Console.Write("Enter Marks in Subject 3: ");
        int m3 = Convert.ToInt32(Console.ReadLine());
        int total = m1 + m2 + m3;
        double average = total / 3.0;
        Console.WriteLine("\nStudent Name : " + name);
        Console.WriteLine("Total Marks : " + total);
        Console.WriteLine("Average    : " + average);
    }
    catch (FormatException)
    {
        Console.WriteLine("\nFormatException: Please enter only integer values
for marks.");
    }
    Console.ReadLine();
}
}

```

OUTPUT

Enter Student Name: Ram

Enter Marks in Subject 1: 70

Enter Marks in Subject 2: 85

Enter Marks in Subject 3: 77

Student Name : Ram

Total Marks : 232

Average : 77.33333333333333

2) In the same Program write a custom exception class called NegativeNumberException if the marks entered are less than zero

using System;

class NegativeNumberException : Exception

{

public NegativeNumberException(string message) : base(message)

{

}

}

class Program

{

static void Main()

{

try

{

Console.Write("Enter Student Name: ");

string name = Console.ReadLine();

Console.Write("Enter Marks in Subject 1: ");

int m1 = Convert.ToInt32(Console.ReadLine());

Console.Write("Enter Marks in Subject 2: ");

```

        int m2 = Convert.ToInt32(Console.ReadLine());
        Console.Write("Enter Marks in Subject 3: ");
        int m3 = Convert.ToInt32(Console.ReadLine());
        if (m1 < 0 || m2 < 0 || m3 < 0)
            throw new NegativeNumberException("Marks cannot be less than
zero.");
        int total = m1 + m2 + m3;
        double average = total / 3.0;
        Console.WriteLine("\nStudent Name : " + name);
        Console.WriteLine("Total Marks : " + total);
        Console.WriteLine("Average    : " + average);
    }
    catch (FormatException)
    {
        Console.WriteLine("FormatException: Please enter only integer values.");
    }
    catch (NegativeNumberException ex)
    {
        Console.WriteLine("NegativeNumberException: " + ex.Message);
    }
    Console.ReadLine();
}
}

```

OUTPUT

Enter Student Name: Ram

Enter Marks in Subject 1: 60

Enter Marks in Subject 2: -5

Enter Marks in Subject 3: 70

NegativeNumberException: Marks cannot be less than zero.

3) Catch the following exceptions for the values entered by the user in the Person class created in previous hands-on.

```
using System;
```

```
class Person
```

```
{
```

```
    public string FirstName;
```

```
    public string LastName;
```

```
    public string Email;
```

```
    public DateTime DateOfBirth;
```

```
    public Person(string firstName, string lastName, string email, DateTime dob)
```

```
    {
```

```
        FirstName = firstName;
```

```
        LastName = lastName;
```

```
        Email = email;
```

```
        DateOfBirth = dob;
```

```
    }
```

```
    public void Display()
```

```
    {
```

```
        Console.WriteLine("\nPerson Details");
```

```

        Console.WriteLine("First Name   : " + FirstName);
        Console.WriteLine("Last Name    : " + LastName);
        Console.WriteLine("Email       : " + Email);
        Console.WriteLine("Date of Birth : " +
DateOfBirth.ToString("dd/MM/yyyy"));
    }
}
class Program
{
    static void Main()
    {
        try
        {
            Console.Write("Enter First Name: ");
            string fname = Console.ReadLine();
            Console.Write("Enter Last Name: ");
            string lname = Console.ReadLine();
            Console.Write("Enter Email: ");
            string email = Console.ReadLine();
            Console.Write("Enter Date of Birth (dd/MM/yyyy): ");
            DateTime dob = Convert.ToDateTime(Console.ReadLine());
            Person p = new Person(fname, lname, email, dob);
            p.Display();
        }
        catch (FormatException)

```



```

    {
        Console.WriteLine("FormatException: Invalid date format.");
    }

    Console.ReadLine();
}
}

```

OUTPUT

Enter First Name: RAM

Enter Last Name: k

Enter Email: ram@gmail.com

Enter Date of Birth (dd/MM/yyyy): xyz

FormatException: Invalid date format.

4) FirstName and LastName should not be Empty and only alphabets should be allowed

using System;

class Person

```

{
    public string FirstName;
    public string LastName;
    public string Email;
    public DateTime DateOfBirth;
    public Person(string firstName, string lastName, string email, DateTime dob)
    {

```

```
        if (string.IsNullOrEmpty(firstName))
            throw new ArgumentException("First Name cannot be empty.");
        if (string.IsNullOrEmpty(lastName))
            throw new ArgumentException("Last Name cannot be empty.");
        foreach (char c in firstName)
        {
            if (!char.IsLetter(c))
                throw new ArgumentException("First Name should contain only alphabets.");
        }
        foreach (char c in lastName)
        {
            if (!char.IsLetter(c))
                throw new ArgumentException("Last Name should contain only alphabets.");
        }
        FirstName = firstName;
        LastName = lastName;
        Email = email;
        DateOfBirth = dob;
    }
    public void Display()
    {
        Console.WriteLine("\nPerson Details");
        Console.WriteLine("First Name   : " + FirstName);
        Console.WriteLine("Last Name    : " + LastName);
    }
}
```

```

        Console.WriteLine("Email      : " + Email);

        Console.WriteLine("Date of Birth : " +
DateOfBirth.ToString("dd/MM/yyyy"));
    }
}

class Program
{
    static void Main()
    {
        try
        {
            Console.Write("Enter First Name: ");
            string fname = Console.ReadLine();

            Console.Write("Enter Last Name: ");
            string lname = Console.ReadLine();

            Console.Write("Enter Email: ");
            string email = Console.ReadLine();

            Console.Write("Enter Date of Birth: ");
            DateTime dob = Convert.ToDateTime(Console.ReadLine());

            Person p = new Person(fname, lname, email, dob);
            p.Display();
        }
        catch (ArgumentException ex)
        {
            Console.WriteLine("Exception: " + ex.Message);
        }
    }
}

```

```
    }  
    catch (FormatException)  
    {  
        Console.WriteLine("Invalid Date Format.");  
    }  
    Console.ReadLine();  
}  
}
```

OUTPUT

Enter First Name: nandu123

Enter Last Name: konda

Enter Email: nandu@gmail.com

Enter Date of Birth: 02/04/2005

Exception: First Name should contain only alphabets.

file handling

1)Create a Console Application to create a file and write contents, save and close the file.

```
using System;  
using System.IO;  
class Program  
{  
    static void Main()  
    {
```

```
try
{
    Console.Write("Enter file path: ");
    string path = Console.ReadLine();
    Console.Write("Enter text to store in file: ");
    string text = Console.ReadLine();
    File.WriteAllText(path, text);
    Console.WriteLine("File created and data saved successfully.");
}
catch (IOException e)
{
    Console.WriteLine("File Error: " + e.Message);
}
catch (Exception e)
{
    Console.WriteLine("Exception: " + e.Message);
}
Console.ReadLine();
}
```

OUTPUT

Enter file path: MyFile.txt

Enter text to store in file: his is my first C# file program

Exception: Access to the path "/home/compiler/MyFile.txt" is denied.